Problem Session #06 Feedback on ERD with Normalization Exercises

สมาชิกกลุ่ม	
กันต์ธีร์ ดวงมณี	67070501003
นัธทวัฒน์ ปริมสิริคุณาวุฒิ	67070501027
ภานุวัฒน์ บุญศักดิ์	67070501034
รัตนนันทน์ สิริพลวัฒน์	67070501038
วิศิษฐ์ สุวรรณเนาว์	67070501042
ศุภวิชญ์ มารยาท	67070501045
พลวริษฐ์ วัฒนเหมรัตน์	67070501067

1. Brainstorming on your ERD

Before Review
CPE241 - Final Project PS04-ERD.pdf

Peer Review
- The schema appears **over-engineered** in certain areas, introducing complexity that exceeds the project's functional requirements. - Some entities do not align well with the core objective of a course-selling platform, potentially impacting model **clarity and maintainability**. - **Content** and **playlist description** support is **limited** and could be improved to better serve presentation and usability needs

New design base on feedback
- The revised ERD better aligns with the project scope by simplifying the overall schema and reducing system complexity. - Removing the **streaming_tokens** table eliminates an implementation-heavy component that was not essential to core functionality. - The removal of the **genres** entity is appropriate for a course-selling platform, with categories providing sufficient content classification. - Adding **Playlist Markdown** and **Content Markdown** enhances content flexibility and supports richer content descriptions.

After review
CPE241 - Final Project PS06-ERD.pdf

Group **Music Streaming**
https://drive.google.com/file/d/1IHPaDCJmQgehtTTTvSkVw3QzITA5464r/view?usp=sharing

Common entities between the two streaming
1. User - Both systems include a user-related entity. - First system: user - Second system: User - Similar functions: Store user account information - Connected to playlists - Connected to purchase/payment records - Connected to activity tracking 2. Playlist - Both systems include a playlist entity. - First system: playlist - Second system: Playlist Purpose: - Allows users to create collections of music/content - Has a relationship with User (1:N) - Has a relationship with Music/Content (M:N through a junction table) 3. Content / Music - Both systems include a main content entity. - First system: music - Second system: Contents Similarities: Represents the main content consumed by users

Associated with genres
Included in playlists
Related to views, progress tracking, and purchases

Difference:
- The first system focuses only on music
- The second system supports multiple content types (Movie / Series / Music)

4. Genre
- First system: genre
- Second system: Genres
- In both systems, Genre is connected to music/content using a Many-to-Many relationship.

5. Playlist–Content Junction Table (M:N)
- First system: playlist_items
- Second system: Playlist Content

Same function:
- Connects Playlist ↔ Content/Music
- Resolves the Many-to-Many relationship

6. Purchase / Transaction
- First system:
transaction
transaction_detail
Second system:
Order
Order Items
Payment
Similar concept:
-Users purchase content
-Separate header and detail tables
-Include status fields

7. Library
-First system: library
-Second system: User Library

- Stores the content owned or saved by users.

8. Activity / View / History
-First system:
-user_activity_log
-listen_history
-Second system:
-Content Views
-Watch Progress

Similar concept:
-Store user behavior data
-Track listening/viewing activities

Common relation between the two streaming

1. User — Playlist

One User → Many Playlists (1:N)

2. Playlist — Content/Music

Many-to-Many (M:N)

Resolved using a junction table

3. Content — Genre

Many-to-Many (M:N)

4. User — Purchase/Order

One User → Many Orders (1:N)

5. Order — Order Items

One Order → Many Items(1:N)

6. User — Library

One User → Many Content items in the library (1:N)

7. User — View/History

One User → Many activity records (1:N)

Group **Empathy เพื่อคนไทย**

Lucidchart

Common entities between the two streaming

1. User / Account

- Both systems include a user-related entity.
- First system: User
- Second system: user
- Similar functions:
    - Store user account information (e.g., name, email, credentials)
    - Connected to personal item collections (Cart / Library)
    - Connected to purchase and payment records

2. Core Product / Item

- Both systems include a main entity representing the core item being offered.
- First system: Product
- Second system: title
- Similar functions:
    - Represent the main content or physical item consumed by users
    - Associated with categories or genres for easy filtering

- Related to user transactions and collections

3. Category / Classification

- Both systems include an entity to group or classify the core items.
- First system: Category
- Second system: genre
- Similar functions:
  - Organize items to help users search and navigate the platform
  - Connected to the core product/title entity (often via a Many-to-Many relationship)

4. Purchase / Transaction

- Both systems include a transaction-related entity.
- First system: Order / Payment
- Second system: purchasement
- Similar functions:
  - Record the history of user payments and purchases
  - Track timestamps (e.g., created_at, payment_verified_at)
  - Link the specific user to the items they bought

5. User's Collection / Saved Items

- Both systems include an entity for users to save or collect specific items.
- First system: Cart / Cart_Item
- Second system: user_library / playlist
- Similar functions:
  - Connect a specific User to specific Products/Titles (resolving M:N relationships)
  - Allow users to store items for future actions (checkout later, or watch later)

Common relation between the two streaming

1. User — Transaction / Purchase

- One User → Many Transactions (1:N)
- First system: User → Order / Payment
- Second system: user → purchasement
- Similar concept: A single user can make multiple purchases or orders over time.

2. Core Item — Category / Genre

- Many-to-Many (M:N)

- First system: Product ↔ Category (typically resolved by a junction table)
- Second system: title ↔ genre (resolved by the title_genre junction table)
- Similar concept: One product/title can belong to multiple categories/genres, and one category/genre contains multiple products/titles.

3. User — Personal Collection

- One User → Many Collections (1:N)
- First system: User → Cart
- Second system: user → playlist / user_library
- Similar concept: A user can create or own multiple collections (like shopping carts or video playlists) linked directly to their account.

4. Personal Collection — Core Item

- Many-to-Many (M:N)
- First system: Cart ↔ Product (resolved by the Cart_Item junction table)
- Second system: playlist ↔ title (resolved by the playlist_title junction table)
- Similar concept: A collection (cart/playlist) contains multiple items, and a single item can be added to multiple users' collections. Both systems use a junction table to resolve this relationship.

2. Normalization exercises

1. Students-Courses-Grades

Before Normalization

Students-Courses-Grades (id, name, phone1, phone2, address 1, address 2,course name, courseDescription, teachers, grade, registerDate, examDate)

- **Table 1 Students-Courses-Grades**

First Name	Last Name	Department Name	GPA	Phone 1	Phone 2	Phone 3	Email Primary	Email 2	Email 3	Course Name	Course Description	Credit	Registration Date	Grade
John	Doe	Computer Science	3.75	1234567890	0987654321	NULL	john.doe@example.com	j.doe@univ.edu	-	Database Systems	Introduction to databases	3	2024-02-17	A
Jane	Smith	Mathematics	3.60	2345678901	8765432109	NULL	jane.smith@example.com	-	-	Linear Algebra	Concepts of vector spaces and matrices	3	2024-02-17	B
Alice	Johnson	Computer Science	3.90	3456789012	NULL	NULL	alice.j@example.com	a.johnson@univ.edu	-	Database Systems	Introduction to databases	3	2024-02-18	A
Bob	Brown	Physics	3.50	4567890123	NULL	NULL	bob.brown@example.com	-	-	Quantum Mechanics	Study of wave-particle duality	3	2024-02-19	C

After Normalization

1NF
The image shows multiple columns for phones (Phone 1, Phone 2, Phone 3) and emails (Email Primary, Email 2, Email 3). We must move these multi-valued attributes into their own tables.

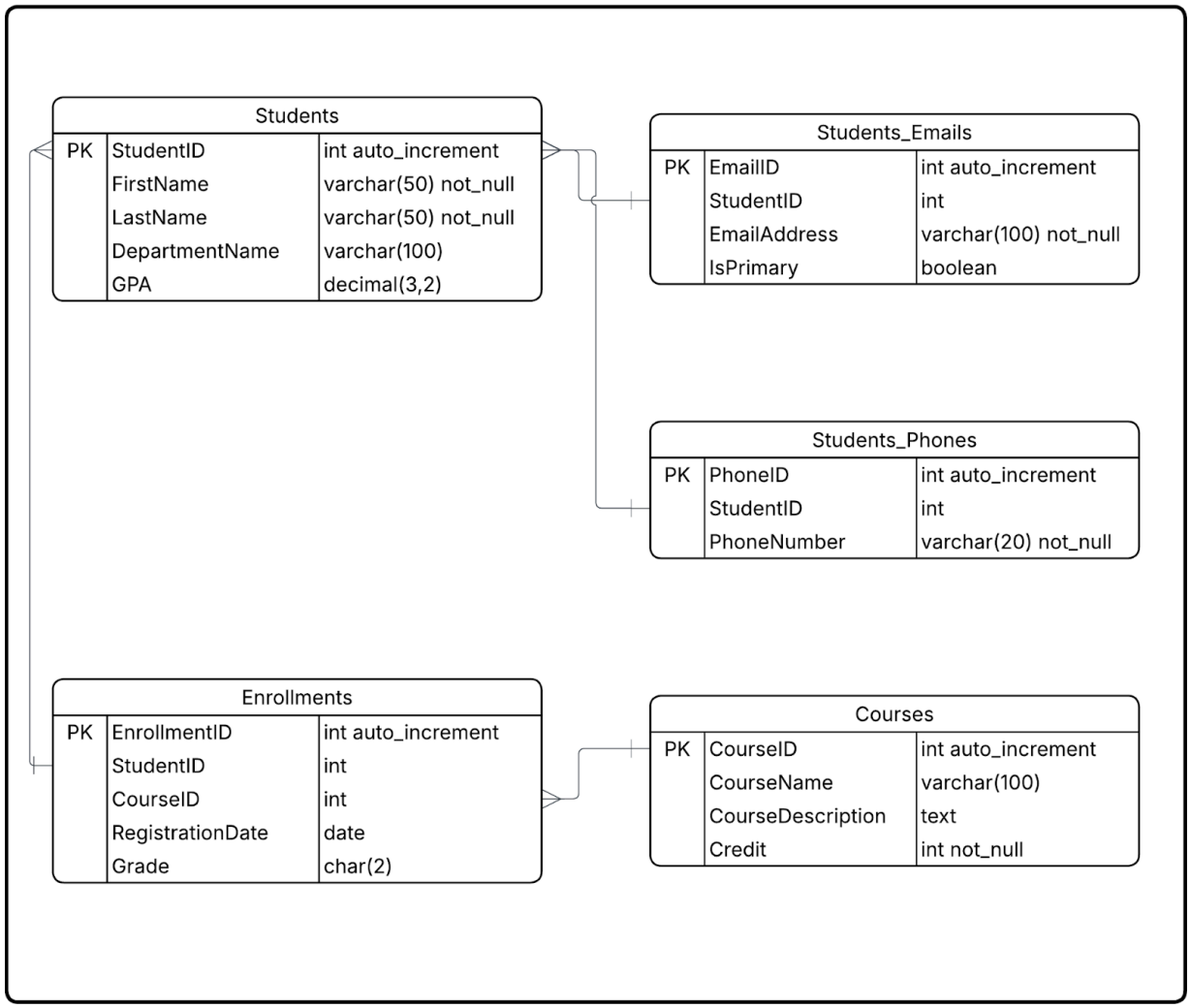
2NF
The core entity here is an "Enrollment" linking a Student to a Course. Course details (Course Name, Course Description, Credit) depend only on the Course, not the Student. Student details depend only on the Student. We break these into Students, Courses, and Enrollments tables.

3NF & BCNF
In the Students table, Department Name and GPA depend directly on the student. (Note: GPA is technically a derived attribute from grades, which strictly violates 3NF, but it is often cached in physical databases for performance. I will keep it as a decimal column but note its derived nature). Every determinant is a candidate key.

```
-- Create Students Table
CREATE TABLE Students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
```

```
    DepartmentName VARCHAR(100),
    GPA DECIMAL(3,2)
);

-- Handle 1NF multi-valued attributes for Contact Info
CREATE TABLE Student_Emails (
    EmailID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    EmailAddress VARCHAR(100) NOT NULL,
    IsPrimary BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);

CREATE TABLE Student_Phones (
    PhoneID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    PhoneNumber VARCHAR(20) NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);

-- Create Courses Table
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY AUTO_INCREMENT,
    CourseName VARCHAR(100) NOT NULL,
    CourseDescription TEXT,
    Credit INT NOT NULL
);

-- Create Junction Table for Enrollments (Grades & Registration)
CREATE TABLE Enrollments (
    EnrollmentID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT,
    CourseID INT,
    RegistrationDate DATE,
    Grade CHAR(2),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

1


Students
PK StudentID int auto_increment
FirstName varchar(50) not_null
LastName varchar(50) not_null
DepartmentName varchar(100)
GPA decimal(3,2)
Students_Emails
PK EmailID int auto_increment
StudentID int
EmailAddress varchar(100) not_null
IsPrimary boolean
Students_Phones
PK PhoneID int auto_increment
StudentID int
PhoneNumber varchar(20) not_null
Enrollments
PK EnrollmentID int auto_increment
StudentID int
CourseID int
RegistrationDate date
Grade char(2)
Courses
PK CourseID int auto_increment
CourseName varchar(100)
CourseDescription text
Credit int not_null

2. Doctor-Patient-Appointment

Before Normalization

Artist-Album-Song (artist, genre, album name, songs, songGenre, songLength)

Doctor Name	Specialty	Office Phone	Mobile Phone	Patient Name	Diagnosis	Treatment	Insurance	Hospital	Building	Floor	Room Number	Appointment Date
Dr. John Smith	Cardiology	123-456-7890	987-654-3210	Alice Johnson	Hypertension	Prescribed medication...	Aetna	City General Hospital	North Wing	3	305A	2024-02-17
Dr. Emily Davis	Dermatology	234-567-8901	876-543-2109	Robert Brown	Eczema	Topical steroid cream	Blue Cross	Metro Medical Center	East Wing	2	204B	2024-02-18
Dr. John Smith	Cardiology	123-456-7890	987-654-3210	Michael Carter	High Cholesterol	Prescribed statins	United Healthcare	City General Hospital	North Wing	3	305A	2024-02-19
Dr. Emily Davis	Dermatology	234-567-8901	876-543-2109	Jessica Miller	Acne	Oral antibiotics and skincare	Cigna	Metro Medical Center	East Wing	2	204B	2024-02-20

After Normalization

1NF
Establish a primary key for the appointments. We will create surrogate keys for entities to ensure uniqueness.

2NF
The unnormalized table merges Doctor info, Patient info, Location info, and Appointment info.

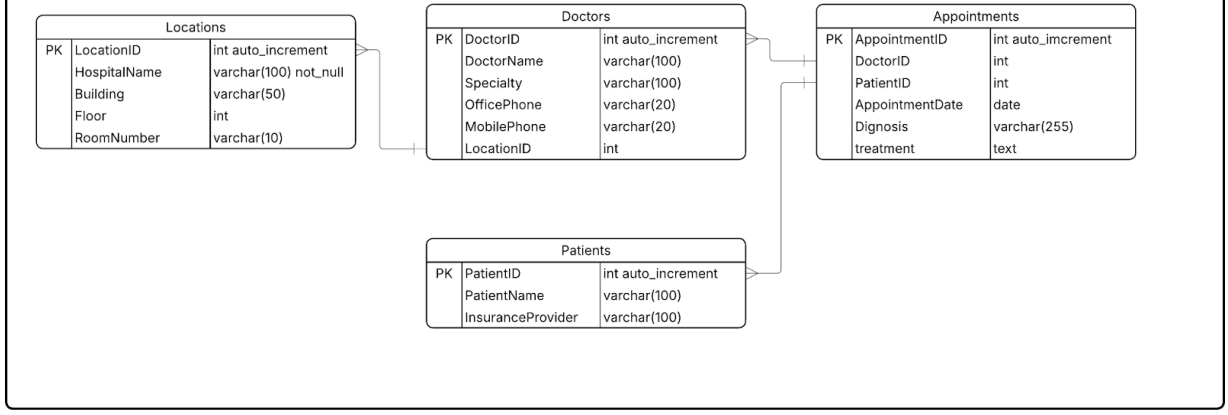
- Doctor details (Specialty, Office Phone, Mobile Phone) depend on the Doctor.
- Patient details (Insurance) depend on the Patient.

3NF & BCNF
Look at the hospital data: Hospital -> Building -> Floor -> Room Number. This is a transitive dependency. A doctor is assigned to a specific location. We should create a Locations table. Furthermore, Diagnosis and Treatment depend entirely on the specific Appointment, so they stay in the junction table.

```
-- Create Locations Table (to resolve transitive hospital dependencies)
CREATE TABLE Locations (
    LocationID INT PRIMARY KEY AUTO_INCREMENT,
    HospitalName VARCHAR(100) NOT NULL,
    Building VARCHAR(50),
    Floor INT,
    RoomNumber VARCHAR(10)
);
```

```sql
-- Create Doctors Table
CREATE TABLE Doctors (
    DoctorID INT PRIMARY KEY AUTO_INCREMENT,
    DoctorName VARCHAR(100) NOT NULL,
    Specialty VARCHAR(100),
    OfficePhone VARCHAR(20),
    MobilePhone VARCHAR(20),
    LocationID INT,
    FOREIGN KEY (LocationID) REFERENCES Locations(LocationID)
);

-- Create Patients Table
CREATE TABLE Patients (
    PatientID INT PRIMARY KEY AUTO_INCREMENT,
    PatientName VARCHAR(100) NOT NULL,
    InsuranceProvider VARCHAR(100)
);

-- Create Appointments Table
CREATE TABLE Appointments (
    AppointmentID INT PRIMARY KEY AUTO_INCREMENT,
    DoctorID INT,
    PatientID INT,
    AppointmentDate DATE,
    Diagnosis VARCHAR(255),
    Treatment TEXT,
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
);
```

2


Locations
PK LocationID int auto_increment
HospitalName varchar(100) not_null
Building varchar(50)
Floor int
RoomNumber varchar(10)
Doctors
PK DoctorID int auto_increment
DoctorName varchar(100)
Specialty varchar(100)
OfficePhone varchar(20)
MobilePhone varchar(20)
LocationID int
Appointments
PK AppointmentID int auto_imcrement
DoctorID int
PatientID int
AppointmentDate date
Dignosis varchar(255)
treatment text
Patients
PK PatientID int auto_increment
PatientName varchar(100)
InsuranceProvider varchar(100)

3. Artist-Album-Song

Before Normalization

Artist-Album-Song (artist, genre, album name, songs, songGenre, songLength)

Artist	Country	Genre	Album Name	Release Year	Record Label	Album Length	Song Title	Track #	Song Genre	Song Length
The Beatles	UK	Rock	Abbey Road	1969	Apple Records	00:47:23	Come Together	1	Rock	00:04:20
The Beatles	UK	Rock	Abbey Road	1969	Apple Records	00:47:23	Something	2	Rock	00:03:02
The Beatles	UK	Rock	Abbey Road	1969	Apple Records	00:47:23	Here Comes the Sun	7	Rock	00:03:06
Taylor Swift	USA	Pop	1989	2014	Big Machine	00:48:41	Blank Space	2	Pop	00:03:51
Taylor Swift	USA	Pop	1989	2014	Big Machine	00:48:41	Shake It Off	6	Pop	00:03:39
Eminem	USA	Hip-Hop	The Eminem Show	2002	Shady Records	01:17:16	Without Me	4	Hip-Hop	00:04:50
Eminem	USA	Hip-Hop	The Eminem Show	2002	Shady Records	01:17:16	Sing for the Moment	12	Hip-Hop	00:05:40

After Normalization

1NF
Ensure all attributes are atomic. The data is largely flat, but we need surrogate keys.

2NF

- Country and Genre (Main Artist Genre) depend strictly on the Artist.
- Release Year and Record Label depend strictly on the Album.
- Track #, Song Title, Song Genre, and Song Length depend strictly on the Song (within an album).

3NF & BCNF
Let's look at Album Length. An album's length is mathematically just the sum of the lengths of all songs on that album. Keeping a derived attribute in a table technically violates 3NF because it creates a transitive dependency (Album Length depends on the Songs, not directly the Album PK). To adhere strictly to 3NF/BCNF, Album Length should be dropped from the schema and calculated dynamically via queries.

```
-- Create Artists Table
CREATE TABLE Artists (
   ArtistID INT PRIMARY KEY AUTO_INCREMENT,
   ArtistName VARCHAR(100) NOT NULL,
   Country VARCHAR(50),
   PrimaryGenre VARCHAR(50)
);
```

```sql
-- Create Albums Table
CREATE TABLE Albums (
    AlbumID INT PRIMARY KEY AUTO_INCREMENT,
    ArtistID INT,
    AlbumName VARCHAR(150) NOT NULL,
    ReleaseYear INT,
    RecordLabel VARCHAR(100),
    -- Album Length is removed to satisfy 3NF (it is a derived SUM of Song Lengths)
    FOREIGN KEY (ArtistID) REFERENCES Artists(ArtistID)
);

-- Create Songs Table
CREATE TABLE Songs (
    SongID INT PRIMARY KEY AUTO_INCREMENT,
    AlbumID INT,
    TrackNumber INT,
    SongTitle VARCHAR(150) NOT NULL,
    SongGenre VARCHAR(50),
    SongLength TIME, -- Assuming standard TIME format or could use INT for total seconds
    FOREIGN KEY (AlbumID) REFERENCES Albums(AlbumID)
);
```

3